

```
# Cell 1: Install all required packages
!pip install --quiet pandas numpy matplotlib seaborn plotly scikit-learn \
    statsmodels sentence-transformers faiss-cpu nltk wordcloud \
    streamlit pyngrok transformers[torch] python-dotenv openai requests

# For VADER sentiment
import nltk
nltk.download('vader_lexicon')
print("✅ All packages installed and VADER downloaded.")
```

```
===== 31.4/31.4 MB 41.5 MB/s eta 0:00:00
===== 10.2/10.2 MB 51.4 MB/s eta 0:00:00
===== 6.9/6.9 MB 82.1 MB/s eta 0:00:00
✅ All packages installed and VADER downloaded.
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
```

```
# Cell 2: Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Cell 3: Create and save your .env file to Google Drive
import os

# --- ⚠️ IMPORTANT: PASTE YOUR KEYS HERE ---
OPENAI_KEY = "sk-proj-uDBz0l0RYEfjDX0EJ8rzRrPie8dnvk0Qu_VjA7_H60doDQGzB3Wrk5koXdjk07PTF0qxfsgFpKT3B1bkFJ14uwmzAd"
OPENROUTER_KEY = "sk-or-v1-f95...f60" # e.g., "sk-or-..."
NGROK_TOKEN = "32nKIUK42PNMesfAS0tMR4tgHsX_QrHwoeQ55Z3xyjWnvcwd" # from your ngrok dashboard

# --- Set the path in your Google Drive ---
secrets_dir = "/content/drive/MyDrive/Colab_Secrets"
os.makedirs(secrets_dir, exist_ok=True)
env_path = os.path.join(secrets_dir, ".env")

# Write the .env file
with open(env_path, "w") as f:
    f.write(f"OPENAI_API_KEY={OPENAI_KEY}\n")
    f.write(f"OPENROUTER_API_KEY={OPENROUTER_KEY}\n")
    f.write(f"NGROK_AUTH_TOKEN={NGROK_TOKEN}\n")

print(f"✅ Your .env file has been saved to: {env_path}")
print("You only need to run this cell once.")
```

✅ Your .env file has been saved to: /content/drive/MyDrive/Colab\_Secrets/.env  
You only need to run this cell once.

```
# Cell 4: Load API keys from Drive into this Colab session's environment
import os
from dotenv import load_dotenv

env_path = "/content/drive/MyDrive/Colab_Secrets/.env"
if os.path.exists(env_path):
    load_dotenv(env_path)
    print("✅ Environment variables loaded from Google Drive.")
    # Check if keys are loaded (optional)
    print(f"    NGROK_AUTH_TOKEN loaded: {bool(os.environ.get('NGROK_AUTH_TOKEN'))}")
    print(f"    OPENAI_API_KEY loaded: {bool(os.environ.get('OPENAI_API_KEY'))}")
else:
    print(f"⚠️ .env file not found at {env_path}. Please run Cell 3 first.")
```

✅ Environment variables loaded from Google Drive.  
NGROK\_AUTH\_TOKEN loaded: True  
OPENAI\_API\_KEY loaded: True

```
# Cell 4.1: (NEW) Verify API Key Loaded in Colab
import os

print("---- Verifying API Keys Loaded in Colab Environment ----")

# Check for OpenRouter Key
or_key = os.environ.get("OPENROUTER_API_KEY", "")
if or_key:
    print(f"✅ OpenRouter Key Found. Starts with: '{or_key[:7]}...' and ends with: '...{or_key[-4:]}'")
else:
    print("❌ ERROR: OpenRouter Key is NOT loaded in the environment.")

# Check for OpenAI Key
oa_key = os.environ.get("OPENAI_API_KEY", "")
```

```

if oa_key:
    print(f"✅ OpenAI Key Found. Starts with: '{oa_key[:7]}...'")
else:
    print(f"❌ ERROR: OpenAI Key is NOT loaded in the environment.")

```

--- Verifying API Keys Loaded in Colab Environment ---

```

✅ OpenRouter Key Found. Starts with: 'sk-or-v...' and ends with: '....f60'
✅ OpenAI Key Found. Starts with: 'sk-proj...'

```

```

# Cell 5: Create Data Directory
import os
DATA_DIR = "/content/data"
os.makedirs(DATA_DIR, exist_ok=True)
print(f"Directory '{DATA_DIR}' created.")

```

Directory '/content/data' created.

```

# Cell 6: Upload all 11 CSV files
from google.colab import files
import os

# Change working directory to /content/data
os.chdir(DATA_DIR)

print(f"Please upload all 11 Olist CSV files to {DATA_DIR}...")
uploaded = files.upload()

print("\n--- Upload Complete ---")
for fn in uploaded.keys():
    print(f"Uploaded file '{fn}' with size {len(uploaded[fn])} bytes")

# Go back to the root content directory
os.chdir('/content')
print(f"\nChanged directory back to /content")

```

Please upload all 11 Olist CSV files to /content/data...

11 files

```

olist_closed_deals_dataset.csv(text/csv) - 171352 bytes, last modified: 10/12/2019 - 100% done
olist_customers_dataset.csv(text/csv) - 9033957 bytes, last modified: 10/1/2021 - 100% done
olist_geolocation_dataset.csv(text/csv) - 61273883 bytes, last modified: 10/1/2021 - 100% done
olist_marketing_qualified_leads_dataset.csv(text/csv) - 703836 bytes, last modified: 10/12/2019 - 100% done
olist_order_items_dataset.csv(text/csv) - 15438671 bytes, last modified: 10/1/2021 - 100% done
olist_order_payments_dataset.csv(text/csv) - 5777138 bytes, last modified: 10/1/2021 - 100% done
olist_order_reviews_dataset.csv(text/csv) - 14451670 bytes, last modified: 10/1/2021 - 100% done
olist_orders_dataset.csv(text/csv) - 17654914 bytes, last modified: 10/1/2021 - 100% done
olist_products_dataset.csv(text/csv) - 2379446 bytes, last modified: 10/1/2021 - 100% done
olist_sellers_dataset.csv(text/csv) - 174703 bytes, last modified: 10/1/2021 - 100% done
product_category_name_translation.csv(text/csv) - 2613 bytes, last modified: 10/1/2021 - 100% done
Saving olist_closed_deals_dataset.csv to olist_closed_deals_dataset.csv
Saving olist_customers_dataset.csv to olist_customers_dataset.csv
Saving olist_geolocation_dataset.csv to olist_geolocation_dataset.csv
Saving olist_marketing_qualified_leads_dataset.csv to olist_marketing_qualified_leads_dataset.csv
Saving olist_order_items_dataset.csv to olist_order_items_dataset.csv
Saving olist_order_payments_dataset.csv to olist_order_payments_dataset.csv
Saving olist_order_reviews_dataset.csv to olist_order_reviews_dataset.csv
Saving olist_orders_dataset.csv to olist_orders_dataset.csv
Saving olist_products_dataset.csv to olist_products_dataset.csv
Saving olist_sellers_dataset.csv to olist_sellers_dataset.csv
Saving product_category_name_translation.csv to product_category_name_translation.csv

```

--- Upload Complete ---

```

Uploaded file 'olist_closed_deals_dataset.csv' with size 171352 bytes
Uploaded file 'olist_customers_dataset.csv' with size 9033957 bytes
Uploaded file 'olist_geolocation_dataset.csv' with size 61273883 bytes
Uploaded file 'olist_marketing_qualified_leads_dataset.csv' with size 703836 bytes
Uploaded file 'olist_order_items_dataset.csv' with size 15438671 bytes
Uploaded file 'olist_order_payments_dataset.csv' with size 5777138 bytes
Uploaded file 'olist_order_reviews_dataset.csv' with size 14451670 bytes
Uploaded file 'olist_orders_dataset.csv' with size 17654914 bytes
Uploaded file 'olist_products_dataset.csv' with size 2379446 bytes
Uploaded file 'olist_sellers_dataset.csv' with size 174703 bytes
Uploaded file 'product_category_name_translation.csv' with size 2613 bytes

```

Changed directory back to /content

```

# Cell 6: Load all CSVs into pandas DataFrames
import pandas as pd
import os

```

```
DATA_DIR = "/content/data"
```

```

# Updated map to include all 11 files
files_map = {

```

```

"orders": "olist_orders_dataset.csv",
"order_items": "olist_order_items_dataset.csv",
"order_payments": "olist_order_payments_dataset.csv",
"order_reviews": "olist_order_reviews_dataset.csv",
"customers": "olist_customers_dataset.csv",
"products": "olist_products_dataset.csv",
"sellers": "olist_sellers_dataset.csv",
"geolocation": "olist_geolocation_dataset.csv",
"products_translation": "product_category_name_translation.csv",
"mql": "olist_marketing_qualified_leads_dataset.csv",
"closed_deals": "olist_closed_deals_dataset.csv"
}

```

```

dfs = {}
for key, fname in files_map.items():
    path = os.path.join(DATA_DIR, fname)
    if os.path.exists(path):
        dfs[key] = pd.read_csv(path, low_memory=False)
        print(f"Loaded {key} -> {path} shape={dfs[key].shape}")
    else:
        print(f"Missing (skipped): {path}")

```

```

# Quick preview
if 'orders' in dfs:
    display(dfs['orders'].head(2))
if 'mql' in dfs:
    display(dfs['mql'].head(2))

```

```

Loaded orders -> /content/data/olist_orders_dataset.csv shape=(99441, 8)
Loaded order_items -> /content/data/olist_order_items_dataset.csv shape=(112650, 7)
Loaded order_payments -> /content/data/olist_order_payments_dataset.csv shape=(103886, 5)
Loaded order_reviews -> /content/data/olist_order_reviews_dataset.csv shape=(99224, 7)
Loaded customers -> /content/data/olist_customers_dataset.csv shape=(99441, 5)
Loaded products -> /content/data/olist_products_dataset.csv shape=(32951, 9)
Loaded sellers -> /content/data/olist_sellers_dataset.csv shape=(3095, 4)
Loaded geolocation -> /content/data/olist_geolocation_dataset.csv shape=(1000163, 5)
Loaded products_translation -> /content/data/product_category_name_translation.csv shape=(71, 2)
Loaded mql -> /content/data/olist_marketing_qualified_leads_dataset.csv shape=(8000, 4)
Loaded closed_deals -> /content/data/olist_closed_deals_dataset.csv shape=(842, 14)

```

|   | order_id                         | customer_id                      | order_status | order_purchase_timestamp | order_approved_at   |
|---|----------------------------------|----------------------------------|--------------|--------------------------|---------------------|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered    | 2017-10-02 10:56:33      | 2017-10-02 10:56:33 |
| 1 | 53cdb2fc8bc7dce0b6741e2150273451 | b0830fb4747a6c6d20dea0b8c802d7ef | delivered    | 2018-07-24 20:41:37      | 2018-07-24 20:41:37 |

  

|   | mql_id                           | first_contact_date | landing_page_id                  | origin      |
|---|----------------------------------|--------------------|----------------------------------|-------------|
| 0 | dac32acd4db4c29c230538b72f8dd87d | 2018-02-01         | 88740e65d5d6b056e0cda098e1ea6313 | social      |
| 1 | 8c18d1de7f67e60dbd64e3c07d7e9d5d | 2017-10-20         | 007f9098284a86ee80ddeb25d53e0af8 | paid_search |

```

# Cell 7: Basic cleaning utilities and unified column naming
def clean_cols(df):
    df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
    return df

for k in list(dfs.keys()):
    if isinstance(dfs[k], pd.DataFrame):
        dfs[k] = clean_cols(dfs[k])

# Convert date columns to datetime
date_cols = {
    "orders": ["order_purchase_timestamp", "order_approved_at", "order_delivered_carrier_date",
               "order_delivered_customer_date", "order_estimated_delivery_date"],
    "order_reviews": ["review_creation_date", "review_answer_timestamp"],
    "mql": ["first_contact_date"],
    "closed_deals": ["won_date"]
}

for k, cols in date_cols.items():
    if k in dfs:
        for c in cols:
            if c in dfs[k].columns:
                dfs[k][c] = pd.to_datetime(dfs[k][c], errors="coerce")

print("✅ Column names cleaned and dates converted.")

```

✅ Column names cleaned and dates converted.

```

# Cell 8: [ENHANCED] Create a master table at the order-item level
if 'order_items' not in dfs or 'orders' not in dfs:
    print("⚠️ Core datasets (order_items, orders) not loaded. Cannot build master table.")

```

```

else:
    # Start with the central order_items table
    master = dfs["order_items"].merge(dfs["orders"], on="order_id", how="left")

    # Aggregate payments (one order can have multiple payment methods)
    if "order_payments" in dfs:
        pay_agg = dfs["order_payments"].groupby("order_id").agg(
            payment_type=('payment_type', 'first'), # Get first payment type
            payment_installments=('payment_installments', 'max'),
            payment_value=('payment_value', 'sum')
        ).reset_index()
        master = master.merge(pay_agg, on="order_id", how="left")

    # Join customers, products, and sellers
    if "customers" in dfs:
        master = master.merge(dfs["customers"], on="customer_id", how="left")
    if "products" in dfs:
        master = master.merge(dfs["products"], on="product_id", how="left")
    if "sellers" in dfs:
        master = master.merge(dfs["sellers"], on="seller_id", how="left")

    # Join reviews (one order can have multiple reviews, get the first one)
    if "order_reviews" in dfs:
        reviews_agg = dfs["order_reviews"].groupby("order_id").agg(
            review_score=('review_score', 'mean'), # Avg score for the order
            review_comment_message=('review_comment_message', 'first') # Get first comment
        ).reset_index()
        master = master.merge(reviews_agg, on="order_id", how="left")

    # Join product category translation
    if "products_translation" in dfs and "product_category_name" in master.columns:
        master = master.merge(dfs["products_translation"], on="product_category_name", how="left")

    # --- [ENHANCEMENT] Join Marketing Funnel Data ---
    # As per schema: mql -> closed_deals (mql_id) | closed_deals -> sellers (seller_id)
    if "closed_deals" in dfs and "mql" in dfs:
        # First, link mql to closed_deals
        mql_deals = dfs["mql"].merge(dfs["closed_deals"], on="mql_id", how="left")

        # Now, link this marketing data to our master table via seller_id
        # We only need one marketing entry per seller
        mql_deals_agg = mql_deals.groupby("seller_id").agg(
            lead_type=('lead_type', 'first'),
            business_segment=('business_segment', 'first'),
            origin=('origin', 'first')
        ).reset_index()

        master = master.merge(mql_deals_agg, on="seller_id", how="left")
        print("✅ Marketing Funnel data merged into master table.")

    print(f"✅ Master table created. Shape: {master.shape}")
    display(master.head(3))

```

✅ Marketing Funnel data merged into master table.

✅ Master table created. Shape: (112650, 38)

|   | order_id                         | order_item_id |                                  | product_id                       | seller_id | sh |
|---|----------------------------------|---------------|----------------------------------|----------------------------------|-----------|----|
| 0 | 00010242fe8c5a6d1ba2dd792cb16214 | 1             | 4244733e06e7ecb4970a6e2683c13e61 | 48436dade18ac8b2bce089ec2a041202 |           |    |
| 1 | 00018f77f2f0320c557190d7a144bdd3 | 1             | e5f2d52b802189ee658865ca93d83a8f | dd7ddc04e1b6c2c614352b383efe2d36 |           |    |
| 2 | 000229ec398224ef6ca0657da4fc703e | 1             | c777355d18b72b67abbeef9df44fd0fd | 5b51032eddd242adc84c38acab88f23d |           |    |

3 rows x 38 columns

# Cell 9: [ENHANCED] Feature engineering

import numpy as np

# Order item total

master["item\_total"] = master["price"] + master["freight\_value"]

# Delivery time features

if "order\_delivered\_customer\_date" in master.columns:

```

    master["delivery_days"] = (master["order_delivered_customer_date"] - master["order_purchase_timestamp"]).dt.days
    master["delivery_delay"] = (master["order_delivered_customer_date"] - master["order_estimated_delivery_date"]).dt.days
    master["delivery_delay"] = master["delivery_delay"].apply(lambda x: max(0, x)) # Only count actual delays

```

# Date derived fields

if "order\_purchase\_timestamp" in master.columns:

```

    master["order_year"] = master["order_purchase_timestamp"].dt.year

```

```

master["order_month_year"] = master["order_purchase_timestamp"].dt.to_period("M").astype(str)

# Repeat-customer flag
if "customer_unique_id" in master.columns:
    cust_counts = master.groupby("customer_unique_id")["order_id"].nunique().to_dict()
    master["cust_order_count"] = master["customer_unique_id"].map(cust_counts)
    master["is_repeat_customer"] = master["cust_order_count"] > 1

# --- [ENHANCEMENT] Customer Lifetime Value (CLV) ---
# We'll do a simple historical CLV (total spend per customer)
if "customer_unique_id" in master.columns:
    clv_df = master.groupby("customer_unique_id")["item_total"].sum().to_dict()
    master["clv"] = master["customer_unique_id"].map(clv_df)

print("✅ Features added: delivery_days, delivery_delay, order_year, is_repeat_customer, clv")
display(master[["order_id", "item_total", "delivery_days", "delivery_delay", "is_repeat_customer", "clv"]].head(

```

✅ Features added: delivery\_days, delivery\_delay, order\_year, is\_repeat\_customer, clv

|   | order_id                         | item_total | delivery_days | delivery_delay | is_repeat_customer | clv    |  |
|---|----------------------------------|------------|---------------|----------------|--------------------|--------|--|
| 0 | 00010242fe8c5a6d1ba2dd792cb16214 | 72.19      | 7.0           | 0.0            | False              | 72.19  |  |
| 1 | 00018f77f2f0320c557190d7a144bdd3 | 259.83     | 16.0          | 0.0            | True               | 284.56 |  |
| 2 | 000229ec398224ef6ca0657da4fc703e | 216.87     | 7.0           | 0.0            | False              | 216.87 |  |
| 3 | 00024acbcd0a6daa1e931b038114c75  | 25.78      | 6.0           | 0.0            | False              | 25.78  |  |
| 4 | 00042b26cf59d7ce69dfabb4e55b4fd9 | 218.04     | 25.0          | 0.0            | False              | 218.04 |  |

# Cell 8.5 – VERIFICATION (Run this AFTER Cell 8 and BEFORE Cell 9)  
import pandas as pd

```

if 'master' in globals() and isinstance(master, pd.DataFrame):
    print("---- Checking required columns in 'master' DataFrame ----")

    # These columns MUST exist after Cell 8 finishes
    required_cols = ['price', 'freight_value', 'customer_unique_id', 'order_id',
                    'review_score', 'review_comment_message', 'product_category_name_english']

    missing_cols = []
    for col in required_cols:
        if col not in master.columns:
            print(f"❌ ERROR: Column '{col}' is MISSING.")
            missing_cols.append(col)
        else:
            print(f"✅ SUCCESS: Column '{col}' is present.")

    if missing_cols:
        print(f"\n⚠️ ERROR: The following columns are missing from 'master': {missing_cols}")
        print("    This means the merge in Cell 8 failed, or the CSVs in Cell 6–7 didn't load correctly.")
        print("    Please check the output of Cell 6 (Upload) and Cell 7 (Load) to ensure all 11 files were loaded")
    else:
        print("\n✅ All required columns are present in 'master'. You can now proceed to Cell 9.")
    else:
        print("❌ 'master' DataFrame not found. Please re-run Cell 8.")

```

--- Checking required columns in 'master' DataFrame ---

```

✅ SUCCESS: Column 'price' is present.
✅ SUCCESS: Column 'freight_value' is present.
✅ SUCCESS: Column 'customer_unique_id' is present.
✅ SUCCESS: Column 'order_id' is present.
✅ SUCCESS: Column 'review_score' is present.
✅ SUCCESS: Column 'review_comment_message' is present.
✅ SUCCESS: Column 'product_category_name_english' is present.

```

✅ All required columns are present in 'master'. You can now proceed to Cell 9.

# Cell 10: Sentiment analysis using VADER  
from nltk.sentiment.vader import SentimentIntensityAnalyzer  
import pandas as pd

```

sia = SentimentIntensityAnalyzer()

def compute_sentiment(text):
    if pd.isna(text):
        return 0.0 # Neutral for blank reviews
    return sia.polarity_scores(str(text))['compound']

if 'review_comment_message' in master.columns:
    master['review_sentiment'] = master['review_comment_message'].apply(compute_sentiment)
    print("✅ Sentiment analysis complete.")
    display(master['review_sentiment'].describe())

```

```
else:
    print("⚠️ 'review_comment_message' column not found for sentiment analysis.")
```

✅ Sentiment analysis complete.

|       | review_sentiment |
|-------|------------------|
| count | 112650.000000    |
| mean  | -0.001025        |
| std   | 0.115587         |
| min   | -0.903900        |
| 25%   | 0.000000         |
| 50%   | 0.000000         |
| 75%   | 0.000000         |
| max   | 0.913600         |

dtype: float64

```
# Cell 11: Simple clustering of customers
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import pandas as pd
```

```
if 'customer_unique_id' in master.columns and 'master' in globals():
```

```
    # Aggregate data for clustering
    cust_agg = master.groupby("customer_unique_id").agg(
        orders_count = ("order_id", "nunique"),
        avg_order_value = ("item_total", "mean"),
        avg_rating = ("review_score", "mean"),
        avg_sentiment = ("review_sentiment", "mean"),
        clv = ("clv", "first")
    ).fillna(0)

    # Standardize and cluster
    scaler = StandardScaler()
    cluster_features = ['orders_count', 'avg_order_value', 'avg_rating', 'clv']

    X = scaler.fit_transform(cust_agg[cluster_features])

    kmeans = KMeans(n_clusters=4, random_state=42, n_init=10).fit(X)
    cust_agg['cluster'] = kmeans.labels_

    # Map clusters back to master table
    master['customer_cluster'] = master['customer_unique_id'].map(cust_agg['cluster'].to_dict())

    print("✅ Customer clustering complete. Cluster counts:")
    print(cust_agg['cluster'].value_counts())
else:
    print("⚠️ Customer data or master DataFrame not available for clustering.")
```

✅ Customer clustering complete. Cluster counts:

| cluster |       |
|---------|-------|
| 3       | 69038 |
| 0       | 20708 |
| 1       | 2885  |
| 2       | 2789  |

Name: count, dtype: int64

```
# Cell 12: Monthly sales forecasting using Exponential Smoothing
```

```
import pandas as pd
import numpy as np
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import matplotlib.pyplot as plt
```

```
# Ensure 'master' DataFrame is available
```

```
if 'master' in globals() and 'order_purchase_timestamp' in master.columns and 'item_total' in master.columns:
```

```
    # Aggregate sales by month
    monthly = master.groupby("order_month_year").agg(
        month_sales=("item_total", "sum")
    ).reset_index()

    # Convert month-year string to datetime
    monthly['ds'] = pd.to_datetime(monthly['order_month_year'] + "-01")
    monthly = monthly.sort_values('ds').set_index('ds')
    monthly = monthly[['month_sales']]

    # Fit Holt-Winters model
```

```

# Fit Holt-Winters model
# We use 'mul' (multiplicative) for trend and seasonality as sales often grow proportionally
model = ExponentialSmoothing(
    monthly['month_sales'],
    trend='mul',
    seasonal='mul',
    seasonal_periods=12 # 12 months in a year
).fit()

# Create future dataframe and forecast for the next 6 months
future_dates = pd.date_range(monthly.index.max() + pd.DateOffset(months=1), periods=6, freq='MS')
forecast_values = model.forecast(6)
forecast_df = pd.DataFrame({'ds': future_dates, 'yhat': forecast_values})

# Plot for sanity check
plt.figure(figsize=(12, 6))
plt.plot(monthly['month_sales'], label='Historical Sales')
plt.plot(forecast_df.set_index('ds')['yhat'], label='6-Month Forecast', linestyle='--')
plt.title("Monthly Sales Forecast (Holt-Winters)")
plt.legend()
plt.grid(True)
plt.savefig("monthly_sales_forecast.png") # Save the plot
plt.show()

# Save forecast data for Streamlit app
forecast_df.to_csv("/content/forecast_monthly.csv", index=False)
print("✅ Forecast plot saved as 'monthly_sales_forecast.png'")
print("✅ Forecast data saved to /content/forecast_monthly.csv")

else:
    print("❌ 'master' DataFrame with 'order_purchase_timestamp' and 'item_total' not found.")

```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is
return get_prediction_index()
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index
return get_prediction_index()

```



```

✅ Forecast plot saved as 'monthly_sales_forecast.png'
✅ Forecast data saved to /content/forecast_monthly.csv

```

```

# Cell 13: [FIXED] Marketing Deal Closing Prediction
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
import joblib
import pandas as pd

if 'mql' in dfs and 'closed_deals' in dfs:
    # 1. Create the target variable
    mql_data = dfs['mql'].copy()

```

```

closed_deals_mql_ids = dfs['closed_deals']['mql_id'].unique()
mql_data['is_closed'] = mql_data['mql_id'].isin(closed_deals_mql_ids).astype(int)

# 2. Select features (These must ONLY come from the mql_data table)
# 'origin' and 'landing_page_id' are the features we know about a new lead.
features = ['origin', 'landing_page_id']
target = 'is_closed'

# Fill NaNs in features before training
mql_data[features] = mql_data[features].fillna('unknown')

# 3. Define preprocessing
categorical_features = ['origin', 'landing_page_id']
# OneHotEncoder will handle the high number of landing_page_id values
categorical_transformer = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[('cat', categorical_transformer, categorical_features)],
    remainder='passthrough'
)

# 4. Create a pipeline
clf = Pipeline(steps=[('preprocessor', preprocessor),
                      ('classifier', LogisticRegression(random_state=42, max_iter=1000))])

# 5. Split and train
X = mql_data[features]
y = mql_data[target]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

clf.fit(X_train, y_train)

# 6. Evaluate
y_pred = clf.predict(X_test)
print("✅ Marketing Deal Closing Model (Logistic Regression) Trained.")
print(f"    Accuracy: {accuracy_score(y_test, y_pred):.2f}")
# Show report (will be imbalanced, but it's a start)
print(classification_report(y_test, y_pred, zero_division=0))

# 7. Save the model for the Streamlit app
joblib.dump(clf, "/content/deal_closing_model.joblib")
print("✅ Deal closing model saved to /content/deal_closing_model.joblib")

else:
    print("⚠️ Marketing datasets (mql, closed_deals) not loaded. Skipping ML model.")

```

✅ Marketing Deal Closing Model (Logistic Regression) Trained.  
Accuracy: 0.90

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.90      | 1.00   | 0.94     | 1432    |
| 1            | 0.00      | 0.00   | 0.00     | 168     |
| accuracy     |           |        | 0.90     | 1600    |
| macro avg    | 0.45      | 0.50   | 0.47     | 1600    |
| weighted avg | 0.80      | 0.90   | 0.85     | 1600    |

✅ Deal closing model saved to /content/deal\_closing\_model.joblib

```

# Cell 14: [ENHANCED] RAG FAISS Index & Embeddings
from sentence_transformers import SentenceTransformer
import faiss
import json
import numpy as np
import pandas as pd

# Ensure 'master' DataFrame is available
if 'master' in globals() and isinstance(master, pd.DataFrame):

    # We will index both product categories and review comments for a richer search
    product_texts = master['product_category_name_english'].dropna().unique().tolist()
    review_texts = master['review_comment_message'].dropna().unique().tolist()

    # Combine and limit for speed in Colab
    # We'll take more reviews than products since they are more descriptive
    rag_texts = (product_texts + review_texts[:4000]) # Limit to ~5k unique entries
    rag_texts = list(set(rag_texts)) # Ensure uniqueness

    print(f"Total unique texts to index for RAG: {len(rag_texts)}")

    if rag_texts:
        embed_model = SentenceTransformer('all-MiniLM-L6-v2')
        embeddings = embed_model.encode(rag_texts, show_progress_bar=True, convert_to_numpy=True)

```



```

# Create FAISS index
d = embeddings.shape[1]
index = faiss.IndexFlatL2(d)
index.add(embeddings.astype('float32')) # FAISS needs float32

# Save index and texts for the app
faiss.write_index(index, "/content/product_review.faiss")
with open("/content/rag_texts.json", "w", encoding="utf-8") as f:
    json.dump(rag_texts, f, ensure_ascii=False)

print("✅ RAG FAISS index and texts saved to /content/")
else:
    print("⚠️ No text data found to build RAG index.")
else:
    print("❌ 'master' DataFrame not found. Please re-run the previous cells.")

```

Total unique texts to index for RAG: 4071  
 /usr/local/lib/python3.12/dist-packages/huggingface\_hub/utils/\_auth.py:94: UserWarning:  
 The secret `HF\_TOKEN` does not exist in your Colab secrets.  
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>)  
 You will be able to reuse this secret in all of your notebooks.  
 Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 33.1kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.93kB/s]
README.md: 10.5k/? [00:00<00:00, 971kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.75kB/s]
config.json: 100% 612/612 [00:00<00:00, 51.1kB/s]
model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 78.4MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 19.7kB/s]
vocab.txt: 232k/? [00:00<00:00, 7.36MB/s]
tokenizer.json: 466k/? [00:00<00:00, 18.8MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 3.68kB/s]
config.json: 100% 190/190 [00:00<00:00, 15.7kB/s]
Batches: 100% 128/128 [00:48<00:00, 10.54it/s]
✅ RAG FAISS index and texts saved to /content/

```

```

%%bash
cat > /content/helper_llm.py <<'PY'
# helper_llm.py
import os
import json
import requests
from dotenv import load_dotenv

# --- [FIX] This script will now be passed the keys directly ---
# We still load .env as a fallback, but the app will pass keys
env_path = "/content/drive/MyDrive/Colab_Secrets/.env"
if os.path.exists(env_path):
    load_dotenv(env_path)

# --- [FIX] The function now accepts keys as arguments ---
def ask_llm(prompt, provider="openai", model=None, max_tokens=1024, temperature=0.0, openai_key="", openrouter_key=""):
    """
    Main function to call the selected LLM provider.
    Now accepts keys directly from the Streamlit app.
    """

    # Use the passed key first, then fall back to environment (which might be empty)
    OPENAI_API_KEY = openai_key or os.getenv("OPENAI_API_KEY", "")
    OPENROUTER_API_KEY = openrouter_key or os.getenv("OPENROUTER_API_KEY", "")

    if provider == "openai":
        if not OPENAI_API_KEY:
            return ("[[LLM NOT CONFIGURED]] OPENAI_API_KEY is not set. "
                    "Please set it in your /content/drive/MyDrive/Colab_Secrets/.env file and restart the notebook")
        try:
            import openai
            client = openai.OpenAI(api_key=OPENAI_API_KEY)
            model_to_use = model or "gpt-4o-mini"

            completion = client.chat.completions.create(

```

```

        model=model_to_use,
        messages=[
            {"role": "system", "content": "You are a helpful data analyst for a Brazilian e-commerce com"},
            {"role": "user", "content": prompt}
        ],
        temperature=temperature,
        max_tokens=max_tokens
    )
    return completion.choices[0].message.content
except Exception as e:
    return f"[OpenAI call error] {e}"

elif provider == "openrouter":
    if not OPENROUTER_API_KEY:
        return ("[LLM NOT CONFIGURED] OPENROUTER_API_KEY is not set. "
                "Please set it in your /content/drive/MyDrive/Colab_Secrets/.env file and restart the notebo")
    try:
        model_to_use = model or "google/gemini-flash-1.5"

        response = requests.post(
            url="https://openrouter.ai/api/v1/chat/completions",
            headers={
                "Authorization": f"Bearer {OPENROUTER_API_KEY}",
                "Content-Type": "application/json",
                "HTTP-Referer": "http://localhost", # Required header
                "X-Title": "Maersk-Project" # Required header
            },
            json={
                "model": model_to_use,
                "messages": [{"role": "user", "content": prompt}],
                "temperature": temperature,
                "max_tokens": max_tokens
            },
            timeout=30
        )
        response.raise_for_status()
        j = response.json()
        return j['choices'][0]['message']['content']
    except Exception as e:
        return f"[OpenRouter call error] {e}"
else:
    return ("[LLM NOT CONFIGURED] Example reply: 'To get top categories in 2018, run:\n"
            "df[df.order_year==2018].groupby('product_category_name_english')['item_total']"
            ".sum().sort_values(ascending=False).head(10)'")

PY

echo "✅ 'helper_llm.py' script has been UPDATED."

```

✅ 'helper\_llm.py' script has been UPDATED.

```

# Cell 15.1: Confirm file creation
import os
if os.path.exists("/content/helper_llm.py"):
    print("✅ 'helper_llm.py' script saved.")
else:
    print("❌ 'helper_llm.py' was NOT created. Please check Cell 15 for errors.")

```

✅ 'helper\_llm.py' script saved.

```

# Cell 16: Save Processed Data for Streamlit App
import pandas as pd

if 'master' in globals() and isinstance(master, pd.DataFrame):
    # Sample the dataframe to keep it light for Colab/Streamlit
    # 50,000 rows is a good balance of data and performance
    sample_size = min(50000, len(master))
    preview_df = master.sample(n=sample_size, random_state=42)

    # Save the sampled DataFrame to a new CSV
    preview_df.to_csv("/content/master_preview.csv", index=False)
    print(f"✅ Saved 'master_preview.csv' (shape: {preview_df.shape}) for the app.")

    # Also save the marketing dataframes for the Marketing Funnel tab
    if 'mql' in dfs:
        dfs['mql'].to_csv("/content/mql_data.csv", index=False)
        print("✅ Saved 'mql_data.csv' for the app.")
    if 'closed_deals' in dfs:
        dfs['closed_deals'].to_csv("/content/closed_deals_data.csv", index=False)
        print("✅ Saved 'closed_deals_data.csv' for the app.")
else:
    print("⚠️ 'master' DataFrame not found. Please re-run the previous data processing cells (8 & 9).")

```

- ✓ Saved 'master\_preview.csv' (shape: (50000, 48)) for the app.
- ✓ Saved 'mql\_data.csv' for the app.
- ✓ Saved 'closed\_deals\_data.csv' for the app.

```
# Cell 17: [FIXED] Write the full Streamlit app.py
%%bash
cat > /content/app.py <<'PY'
import streamlit as st
import pandas as pd
import numpy as np
import os
import json
import plotly.express as px
import plotly.graph_objects as go
import joblib
from helper_llm import ask_llm
import warnings
import re
import sys

# --- Page Config ---
st.set_page_config(
    layout="wide",
    page_title="Maersk E-Commerce Agent",
    initial_sidebar_state="expanded"
)
warnings.filterwarnings('ignore')

# --- [FIX] Load API keys ---
# We will still try to load them just in case they work later.
OAI_KEY = os.environ.get("OPENAI_API_KEY", "")
OR_KEY = os.environ.get("OPENROUTER_API_KEY", "")
try:
    # This will fail if not run from streamlit run cmd, which is fine
    OAI_KEY = sys.argv[1] or OAI_KEY
    OR_KEY = sys.argv[2] or OR_KEY
except IndexError:
    pass # Use the os.environ keys

# --- CSS ---
css = """
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap');
html, body, [class*="css"] { font-family: 'Inter', sans-serif; }
.stApp { background: #f0f4f8; }
[data-testid="stSidebar"] { background: #002B5B; color: #ffffff; }
[data-testid="stSidebar"] .stMetricLabel { color: #a0c8e0; }
[data-testid="stSidebar"] .stMetricValue { color: #ffffff; font-size: 1.75rem; }
.app-title { font-size: 2.5rem; font-weight: 700; color: #002B5B; padding-top: 1rem; }
.app-sub { font-size: 1.1rem; color: #555; margin-bottom: 1rem; }
.dashboard-card-title { font-size: 1.25rem; font-weight: 600; color: #002B5B; margin-bottom: 10px; }
[data-testid="stChatMessage"] * { color: #000000 !important; }
.stSelectbox div[data-baseweb="select"] > div { color: #000000 !important; }
"""
st.markdown(f"<style>{css}</style>", unsafe_allow_html=True)

# --- Data Loading ---
@st.cache_data(ttl=3600)
def load_data(path):
    if not os.path.exists(path):
        return None
    df = pd.read_csv(path, low_memory=False)
    date_cols = ['order_purchase_timestamp', 'order_delivered_customer_date', 'order_estimated_delivery_date', ]
    for col in date_cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce')
    return df

@st.cache_resource
def load_model(path):
    if not os.path.exists(path):
        return None
    return joblib.load(path)

@st.cache_resource
def load_rag_index():
    if not (os.path.exists("/content/product_review.faiss") and os.path.exists("/content/rag_texts.json")):
        return None, None
    try:
        import faiss
        index = faiss.read_index("/content/product_review.faiss")
        with open("/content/rag_texts.json", "r", encoding="utf-8") as f:
            texts = json.load(f)
    
```

```

        return index, texts
    except Exception as e:
        print(f"Error loading RAG index: {e}")
        return None, None

@st.cache_resource
def load_embed_model():
    from sentence_transformers import SentenceTransformer
    return SentenceTransformer('all-MiniLM-L6-v2')

master_df = load_data("/content/master_preview.csv")
forecast_df = load_data("/content/forecast_monthly.csv")
mql_df = load_data("/content/mql_data.csv")
closed_df = load_data("/content/closed_deals_data.csv")
closing_model = load_model("/content/deal_closing_model.joblib")
rag_index, rag_texts = load_rag_index()
embed_model = load_embed_model()

def get_recommendations(product_category, df, top_n=5):
    if 'product_category_name_english' not in df.columns:
        return pd.DataFrame()
    recs = df[df['product_category_name_english'] == product_category]
    if recs.empty:
        return pd.DataFrame()
    recs = recs.sort_values(by='review_score', ascending=False)
    recs = recs.drop_duplicates(subset=['product_id'])
    return recs[['product_id', 'product_category_name_english', 'price', 'review_score']].head(top_n)

# --- Sidebar ---
with st.sidebar:
    st.image("https://upload.wikimedia.org/wikipedia/commons/thumb/e/e6/Maersk_Group_Logo.svg/1280px-Maersk_Grou")
    mode = st.radio("Select Dashboard",
        [
            "🗯️ Agentic Chat",
            "📊 Analytics Dashboard",
            "📈 Sales & ML Forecasts",
            "📊 Marketing Funnel",
            "🔍 RAG Knowledge Search"
        ])
    st.markdown("----")
    if master_df is not None:
        st.metric("Total Orders (Sample)", f"{master_df['order_id'].nunique():,}")
        st.metric("Total Customers (Sample)", f"{master_df['customer_unique_id'].nunique():,}")
        st.metric("Total Revenue (Sample)", f"R${master_df['item_total'].sum():,.0f}")
    else:
        st.error("master_preview.csv not found!")

# --- Main App Body ---

if mode == "🗯️ Agentic Chat":
    st.markdown("<div class='app-title'>E-Commerce Insights Agent</div>", unsafe_allow_html=True)
    st.markdown("<div class='app-sub'>Ask a question in plain English. The agent can answer questions, run analy")

    provider = st.selectbox("Select LLM Provider (Requires API Key)", ["none (Use hardcoded rules)", "openai", ""])

    if "messages" not in st.session_state:
        st.session_state.messages = [{"role": "assistant", "content": "How can I help you analyze the e-commerce"}

    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    if prompt := st.chat_input("e.g., What are the top 5 product categories by sales?"):
        st.session_state.messages.append({"role": "user", "content": prompt})
        with st.chat_message("user"):
            st.markdown(prompt)

        with st.chat_message("assistant"):
            response = ""
            q_lower = prompt.lower()
            df = master_df

            try:
                if df is None:
                    response = "Sorry, the master dataset is not loaded."

            # --- [NEW] HARDCODED RULE SET ---
            elif "top" in q_lower and "categor" in q_lower:
                n = 5
                if "top 10" in q_lower: n = 10
                if "top 3" in q_lower: n = 3
                top_cats = df.groupby('product_category_name_english')['item_total'].sum().nlargest(n)
                response = f"Here are the top {n} categories by sales:\n\n{top_cats.to_string()}"
                st.dataframe(top_cats)

```

```

elif "qual é a categoria de produto mais vendida" in q_lower:
    top_cat = df.groupby('product_category_name_english')['item_total'].sum().idxmax()
    response = f"A categoria de produto mais vendida é: **{top_cat}**"

elif "recommend" in q_lower and "product" in q_lower:
    match = re.search(r'\b(in|for)\b\s+([\']?[\w\s_]+[\']?)', q_lower)
    category_name = ""
    if match:
        category_name = match.group(2).strip().strip("'").replace(" ", "_")
        all_cats = df['product_category_name_english'].dropna().unique()
        matched_cat = next((c for c in all_cats if category_name == c.lower()), None)
        if matched_cat:
            recs = get_recommendations(matched_cat, df)
            response = f"Here are some top-rated recommendations for '{matched_cat}':\n\n{recs.to_string()}\n\n{st.dataframe(recs)}"
        else:
            response = f"Sorry, I couldn't find a category matching '{category_name}'. Try 'health_b"

elif "how many unique sellers" in q_lower:
    count = df['seller_id'].nunique()
    response = f"There are **{count}** unique sellers in the sample."

elif "cities" in q_lower and "most orders" in q_lower:
    n = 5
    if "10" in q_lower: n = 10
    top_cities = df.groupby('customer_city')['order_id'].nunique().nlargest(n)
    response = f"Here are the top {n} cities by order count:\n\n{top_cities.to_string()}\n\n{st.dataframe(top_cities)}"

elif "average review score" in q_lower and "payment type" in q_lower:
    avg_scores = df.groupby('payment_type')['review_score'].mean()
    response = f"Here is the average review score per payment type:\n\n{avg_scores.to_string()}\n\n{st.dataframe(avg_scores)}"

elif "total revenue for 2017" in q_lower:
    revenue = df[df['order_year'] == 2017]['item_total'].sum()
    response = f"The total revenue for 2017 in the sample is: **R${revenue:,.2f}**"

elif "worst-rated product categories" in q_lower:
    n = 3
    if "5" in q_lower: n = 5
    worst_cats = df.groupby('product_category_name_english')['review_score'].mean().nsmallest(n)
    response = f"Here are the bottom {n} worst-rated categories:\n\n{worst_cats.to_string()}\n\n{st.dataframe(worst_cats)}"

elif "sellers" in q_lower and "highest total revenue" in q_lower:
    n = 10
    if "5" in q_lower: n = 5
    top_sellers = df.groupby('seller_id')['item_total'].sum().nlargest(n)
    response = f"Here are the top {n} sellers by total revenue:\n\n{top_sellers.to_string()}\n\n{st.dataframe(top_sellers)}"

elif "average price" in q_lower and "sports_leisure" in q_lower:
    avg_price = df[df['product_category_name_english'] == 'sports_leisure']['price'].mean()
    response = f"The average price for 'sports_leisure' products is: **R${avg_price:,.2f}**"

elif "cuántos pedidos" in q_lower and "2018" in q_lower:
    count = df[df['order_year'] == 2018]['order_id'].nunique()
    response = f"Hubo **{count}** pedidos únicos en 2018 (en la muestra)."
```

```

elif "délai de livraison moyen" in q_lower:
    avg_delivery = df['delivery_days'].mean()
    response = f"Le délai de livraison moyen est de **{avg_delivery:.1f}** jours (dans l'échanti"

elif "clv" in q_lower or "customer lifetime value" in q_lower:
    avg_clv = df.drop_duplicates(subset=['customer_unique_id'])['clv'].mean()
    response = f"The average Customer Lifetime Value (CLV) from the sample is: R${avg_clv:,.2f}"

elif "delivery" in q_lower and ("delay" in q_lower or "late" in q_lower):
    avg_delay = df['delivery_delay'].mean()
    response = f"The average delivery delay (for late orders) is {avg_delay:.2f} days."

# --- [END] HARDCODED RULE SET ---

else:
    # --- Fallback to LLM ---
    if provider == "none (Use hardcoded rules)":
        response = "Sorry, I can't answer that with my built-in rules. Please select an LLM prov"
    else:
        with st.spinner(f"Asking {provider} agent..."):
            cols_list = ', '.join(df.columns.tolist())

```

```

cats_list = ', '.join(df['product_category_name_english'].dropna().unique()[:10])

full_prompt = f"""
You are an expert data analyst for Olist, a Brazilian e-commerce company.
Your goal is to answer the user's question concisely.

The user's question is: "{prompt}"

If the question can be answered with a simple fact, provide it.
If the question requires data analysis, you MUST provide a single, executable line o
using a dataframe named 'df'.

DO NOT provide explanations, just the code.
DO NOT use backticks (`python ... `).

Here are the columns in the 'df' dataframe:
{cols_list}

Here are some example product categories:
{cats_list}

Example User Question: What are the bottom 5 cities by sales?
Example Answer: df.groupby('customer_city')['item_total'].sum().nsmallest(5)

Example User Question: How many unique customers are there?
Example Answer: df['customer_unique_id'].nunique()
"""
response = ask_llm(full_prompt,
                    provider=provider,
                    openai_key=OAI_KEY,
                    openrouter_key=OR_KEY)

st.markdown(response)

if "df." in response and provider != "none (Use hardcoded rules)":
    try:
        result = eval(response, {"df": df, "pd": pd, "np": np})
        st.markdown("----")
        st.markdown("**Agent Execution Result:**")
        if isinstance(result, pd.DataFrame):
            st.dataframe(result)
        elif isinstance(result, pd.Series):
            st.dataframe(result)
        else:
            st.write(result)
    except Exception as e:
        st.error(f"🚨 Agent's code failed to execute: {e}")

except Exception as e:
    response = f"An error occurred: {e}"
    st.error(response)

st.session_state.messages.append({"role": "assistant", "content": response})

elif mode == "📊 Analytics Dashboard":
    st.markdown("<div class='app-title'>Analytics Dashboards</div>", unsafe_allow_html=True)
    st.markdown("<div class='app-sub'>High-level KPIs and interactive charts for core business areas.</div>", un

if master_df is not None:
    tab1, tab2, tab3 = st.tabs(["Product Analytics", "Order & Delivery", "Customer Insights"])

    with tab1:
        col1, col2 = st.columns(2)
        with col1, st.container(border=True):
            st.markdown("<div class='dashboard-card-title'>Top 10 Categories by Revenue</div>", unsafe_allow
            top_cats = master_df.groupby('product_category_name_english')['item_total'].sum().nlargest(10).r
            fig = px.bar(top_cats, x='product_category_name_english', y='item_total', labels={'item_total':
            st.plotly_chart(fig, use_container_width=True)
        with col2, st.container(border=True):
            st.markdown("<div class='dashboard-card-title'>Top 10 Categories by Rating</div>", unsafe_allow
            topRated_cats = master_df.groupby('product_category_name_english')['review_score'].mean().nlarg
            fig = px.bar(topRated_cats, x='product_category_name_english', y='review_score', labels={'revie
            st.plotly_chart(fig, use_container_width=True)

    with tab2:
        col1, col2 = st.columns(2)
        with col1, st.container(border=True):
            st.markdown("<div class='dashboard-card-title'>Payment Method Distribution</div>", unsafe_allow
            pay_type = master_df['payment_type'].value_counts().reset_index()
            fig = px.pie(pay_type, names='payment_type', values='count', hole=0.3)
            st.plotly_chart(fig, use_container_width=True)
        with col2, st.container(border=True):

```

```

st.markdown("<div class='dashboard-card-title'>Delivery Performance</div>", unsafe_allow_html=True)
avg_delay = master_df['delivery_delay'].mean()
avg_delivery = master_df['delivery_days'].mean()
st.metric("Avg. Delivery Time (Days)", f"{avg_delivery:.1f}")
st.metric("Avg. Delay (if late)", f"{avg_delay:.1f}")
fig = px.histogram(master_df, x='delivery_days', nbins=50, title='Delivery Time Distribution (Days)')
st.plotly_chart(fig, use_container_width=True)

with tab3:
    col1, col2 = st.columns(2)
    with col1, st.container(border=True):
        st.markdown("<div class='dashboard-card-title'>Top 10 Cities by Revenue</div>", unsafe_allow_html=True)
        top_cities = master_df.groupby('customer_city')['item_total'].sum().nlargest(10).reset_index()
        fig = px.bar(top_cities, x='customer_city', y='item_total')
        st.plotly_chart(fig, use_container_width=True)
    with col2, st.container(border=True):
        st.markdown("<div class='dashboard-card-title'>Customer Segments</div>", unsafe_allow_html=True)
        if 'customer_cluster' in master_df.columns:
            cluster_counts = master_df['customer_cluster'].value_counts().reset_index()
            fig = px.pie(cluster_counts, names='customer_cluster', values='count', hole=0.3)
            st.plotly_chart(fig, use_container_width=True)
        else:
            st.info("Customer clusters not computed.")
    else:
        st.error("Master data not loaded. Please run the Colab notebook.")

elif mode == "📊 Sales & ML Forecasts":
    st.markdown("<div class='app-title'>Sales & ML Forecasts</div>", unsafe_allow_html=True)
    st.markdown("<div class='app-sub'>Predictive models for sales and marketing deal closures.</div>", unsafe_allow_html=True)

    with st.container(border=True):
        st.markdown("<div class='dashboard-card-title'>Monthly Sales Forecast</div>", unsafe_allow_html=True)
        if forecast_df is not None:
            forecast_df['ds'] = pd.to_datetime(forecast_df['ds'])
            fig = go.Figure()
            fig.add_trace(go.Scatter(x=forecast_df['ds'], y=forecast_df['yhat'], mode='lines+markers', name='Forecast'))
            fig.update_layout(title='6-Month Sales Forecast (Holt-Winters)', xaxis_title='Date', yaxis_title='Forecast')
            st.plotly_chart(fig, use_container_width=True)
        else:
            st.warning("forecast_monthly.csv not found.")

    st.markdown("----")

    with st.container(border=True):
        st.markdown("<div class='dashboard-card-title'>Marketing Deal Closing Predictor</div>", unsafe_allow_html=True)
        if closing_model is not None and mql_df is not None:
            col1, col2 = st.columns(2)
            with col1:
                origin = st.selectbox("Lead Origin", mql_df['origin'].dropna().unique())
                landing_page = st.selectbox("Landing Page ID", mql_df['landing_page_id'].dropna().unique()[:20])

            with col2:
                input_df = pd.DataFrame([origin, landing_page], columns=['origin', 'landing_page_id'])
                prediction = closing_model.predict(input_df)[0]
                prediction_proba = closing_model.predict_proba(input_df)[0]

                if prediction == 1:
                    st.success(f"***Likely to Close!*** (Probability: {prediction_proba[1]:.1%})")
                else:
                    st.error(f"***Unlikely to Close!*** (Probability: {prediction_proba[0]:.1%})")
                st.caption("This model predicts if a new marketing lead will become a paying seller.")
            else:
                st.warning("Marketing ML model or MQL data not found.")

elif mode == "📈 Marketing Funnel":
    st.markdown("<div class='app-title'>Marketing Funnel Analysis</div>", unsafe_allow_html=True)
    st.markdown("<div class='app-sub'>Analysis of the Marketing Qualified Leads (MQLs) and deal closures.</div>", unsafe_allow_html=True)

    if mql_df is not None and closed_df is not None:
        col1, col2, col3 = st.columns(3)
        with col1:
            st.metric("Total MQLs", f"{mql_df['mql_id'].nunique():,}")
        with col2:
            st.metric("Closed Deals", f"{closed_df['mql_id'].nunique():,}")
        with col3:
            rate = closed_df['mql_id'].nunique() / mql_df['mql_id'].nunique()
            st.metric("MQL-to-Close Rate", f"{rate:.2%}")

        c1, c2 = st.columns(2)
        with c1, st.container(border=True):
            st.markdown("<div class='dashboard-card-title'>MQL by Origin</div>", unsafe_allow_html=True)
            origin_counts = mql_df['origin'].value_counts().reset_index()

```

```

fig = px.bar(origin_counts, x='origin', y='count')
st.plotly_chart(fig, use_container_width=True)
with c2, st.container(border=True):
    st.markdown("<div class='dashboard-card-title'>Top 10 Business Segments (Closed Deals)</div>", unsafe_
segment_counts = closed_df['business_segment'].value_counts().reset_index()
fig = px.bar(segment_counts.head(10), x='business_segment', y='count')
st.plotly_chart(fig, use_container_width=True)
else:
    st.error("Marketing funnel CSVs not found. Please re-run Cell 16.")

elif mode == "🔍 RAG Knowledge Search":
    st.markdown("<div class='app-title'>RAG Knowledge Search</div>", unsafe_allow_html=True)
    st.markdown("<div class='app-sub'>Search for products or review topics using semantic search (powered by FAI

query = st.text_input("Search query", placeholder="e.g., 'comfortable office furniture' or 'late delivery co

if st.button("Search") and query.strip():
    if rag_index is not None and rag_texts is not None and embed_model is not None:
        with st.spinner("Embedding query and searching FAISS index..."):
            import faiss

            query_embedding = embed_model.encode([query]).astype('float32')
            D, I = rag_index.search(query_embedding, k=5)

            st.markdown("<div class='dashboard-card-title'>Top 5 Matching Results:</div>", unsafe_allow_html
            for i in I[0]:
                with st.container(border=True):
                    st.write(rag_texts[i])
        else:
            st.error("RAG index files or model not found. Please run Cell 14.")
PY

echo "✅ 'app.py' script has been UPDATED"

```

✅ 'app.py' script has been UPDATED

```

# Cell 18: Launch Streamlit with ngrok
import os
import sys
import time
import subprocess
from pyngrok import ngrok, conf

# --- 1. Kill any old processes ---
!pkill -f streamlit || true
!pkill -f ngrok || true
time.sleep(2)

# --- 2. Set ngrok Auth Token ---
auth_token = os.environ.get("NGROK_AUTH_TOKEN", "")
if not auth_token:
    print("⚠️ NGROK_AUTH_TOKEN not set. Tunnel may be rate-limited or fail.")
else:
    conf.get_default().auth_token = auth_token
    print("✅ ngrok auth token set.")

# --- 3. Get API keys from notebook environment ---
openai_key = os.environ.get("OPENAI_API_KEY", "NO_OAI_KEY")
openrouter_key = os.environ.get("OPENROUTER_API_KEY", "NO_OR_KEY")

if openai_key == "NO_OAI_KEY" or openrouter_key == "NO_OR_KEY":
    print("⚠️ API keys not found in os.environ. LLM features will fail.")
    print(" (This is OK if you only use the hardcoded rules)")
else:
    print("✅ API keys found in notebook environment.")

# --- 4. Open log files ---
stdout_log = open("/content/streamlit_stdout.log", "w")
stderr_log = open("/content/streamlit_stderr.log", "w")

# --- 5. Start Streamlit and PASS KEYS as arguments ---
cmd = [
    sys.executable, "-m", "streamlit", "run", "/content/app.py",
    "--server.port", "8501",
    "--server.enableCORS", "false",
    "--client.showErrorDetails", "false",
    "--", # This separator tells streamlit the following are app arguments
    openai_key,
    openrouter_key
]

proc = subprocess.Popen(cmd, stdout=stdout_log, stderr=stderr_log)

```



```

print(f"🚀 Starting Streamlit (PID: {proc.pid})... please wait 10-15 seconds.")
time.sleep(10)

# --- 6. Check if Streamlit failed ---
poll_status = proc.poll()
if poll_status is not None:
    print(f"❌ Streamlit failed to start (exit code {poll_status}). Checking logs:")
    !echo "---- STDERR ----"
    !tail -n 30 /content/streamlit_stderr.log
    !echo "---- STDOUT ----"
    !tail -n 30 /content/streamlit_stdout.log
else:
    # --- 7. If running, start ngrok ---
    print(f"✅ Streamlit is running. Starting ngrok tunnel...")
    try:
        public_url = ngrok.connect(addr="8501", bind_tls=True)
        print("=====")
        print(f"🚀 YOUR APP IS LIVE! (FIXED) 🚀")
        print(f"🌐 Public URL: {public_url}")
        print("=====")
    except Exception as e:
        print(f"❌ Failed to start ngrok tunnel: {e}")
        !echo "---- NGROK ERRORS ----"
        !tail -n 30 /content/streamlit_stderr.log

```

^C

^C

✅ ngrok auth token set.

✅ API keys found in notebook environment.

🚀 Starting Streamlit (PID: 10015)... please wait 10-15 seconds.

✅ Streamlit is running. Starting ngrok tunnel...

=====

🚀 YOUR APP IS LIVE! (FIXED) 🚀

🌐 Public URL: NgrokTunnel: "<https://a8a7fae8ff2b.ngrok-free.app>" -> "<http://localhost:8501>"

=====